Практическая работа №2. Выполнение SQL-запросов из C# с помощью SqlCommand

Цель работы

Научиться выполнять SQL-команды из C#-приложения с помощью `SqlCommand` и правильно выбирать метод выполнения запроса:

- `ExecuteNonQuery()`;
- `ExecuteScalar()`;
- `ExecuteReader()`.

В ходе работы необходимо создать небольшое консольное приложение администратора кофейни, которое получает информацию из базы `coffee_shop` и изменяет данные в ней.

Задание 1. Создание проекта

Создайте консольное приложение:

```
dotnet new console -n AdoNetPractice02
```

Перейдите в каталог проекта:

```
cd AdoNetPractice02
```

Установите пакет:

```
dotnet add package Microsoft.Data.SqlClient
```

Проверьте, что проект запускается:

```
dotnet run
```

Задание 2. Подключение к базе данных

Добавьте:

```
using Microsoft.Data.SqlClient;
```

Создайте строку подключения:

```
string connectionString =
    "Server=localhost;Database=coffee_shop;Trusted_Connection
=True;TrustServerCertificate=True;";
```

Создайте соединение:

```
using var connection = new SqlConnection(connectionString);

connection.Open();

Console.WriteLine("Подключение к базе данных установлено.");
```

Запустите программу и убедитесь, что соединение устанавливается без ошибок.

Задание 3. Получение названия базы данных

Создайте SQL-запрос:

```
SELECT DB_NAME();
```

Создайте объект `SqlCommand`:

```
string sql = "SELECT DB_NAME();";

using var command = new SqlCommand(sql, connection);
```

Получите результат:

```
string databaseName =
    Convert.ToString(command.ExecuteScalar())!;

Console.WriteLine($"База данных: {databaseName}");
```

Ожидаемый результат:

```
Подключение к базе данных установлено.
База данных: coffee_shop
```

Обратите внимание, что здесь используется:

```
ExecuteScalar()
```

поскольку запрос возвращает одно значение.

Задание 4. Получение количества товаров

Теперь необходимо узнать, сколько товаров находится в кофейне.

SQL-запрос:

```
SELECT COUNT(*)
```

```
FROM products;
```

Реализуйте его через `SqlCommand`.

Пример:

```
string sql = "SELECT COUNT(*) FROM products;";

using var command = new SqlCommand(sql, connection);

int productCount =
    Convert.ToInt32(command.ExecuteScalar());

Console.WriteLine($"Количество товаров: {productCount}");
```

Убедитесь, что программа выводит корректное количество товаров.

Задание 5. Получение информации о товарах

Теперь необходимо получить список товаров.

Используйте запрос:

```
SELECT id, name, price
FROM products;
```

Для выполнения запроса используйте:

```
ExecuteReader()
```

Пример:

```
string sql = """
    SELECT id, name, price
    FROM products;
```

```
    """;

using var command = new SqlCommand(sql, connection);

using var reader = command.ExecuteReader();

while (reader.Read())
{
    Console.WriteLine(
        $"{reader["id"]}. {reader["name"]} — {reader["pric
e"]} руб."
    );
}
```

Пример результата:

```
1. Эспрессо — 120.00 руб.
2. Американо — 140.00 руб.
3. Капучино — 190.00 руб.
...
```

Обратите внимание:

`ExecuteReader()` используется для получения нескольких строк.

Задание 6. Добавление нового товара

Теперь приложение должно уметь добавлять новый товар.

Для начала используйте фиксированные значения:

```sql
INSERT INTO products (name, category_id, price)
VALUES (N'Мокка', 1, 240.00);
```

В C#:

```
string sql = """
    INSERT INTO products (name, category_id, price)
    VALUES (N'Мокка', 1, 240.00);
    """;

using var command = new SqlCommand(sql, connection);

int rowsAffected = command.ExecuteNonQuery();

Console.WriteLine($"Добавлено строк: {rowsAffected}");
```

После выполнения проверьте базу данных.

В таблице `products` должен появиться новый товар.

Обратите внимание:

```
ExecuteNonQuery()
```

не возвращает данные из таблицы.

Он возвращает количество изменённых строк.

Задание 7. Изменение цены товара

Измените цену добавленного товара.

Например:

```
UPDATE products
SET price = 250.00
WHERE name = N'Мокка';
```

Выполните запрос через:

```
ExecuteNonQuery()
```

После выполнения выведите количество изменённых строк:

```
Изменено строк: 1
```

Проверьте результат в базе данных.

Задание 8. Получение максимальной цены

Теперь определите стоимость самого дорогого товара.

Используйте:

```sql
SELECT MAX(price)
FROM products;
```

Выполните запрос через:

```
ExecuteScalar()
```

Результат необходимо сохранить в переменную типа `decimal`.

Например:

```
decimal maxPrice =
    Convert.ToDecimal(command.ExecuteScalar());

Console.WriteLine($"Максимальная цена: {maxPrice} руб.");
```

Задание 9. Получение количества заказов

Теперь используем другую таблицу.

Необходимо определить количество заказов:

```sql
SELECT COUNT(*)
FROM orders;
```

Выполните запрос через `ExecuteScalar()`.

Результат выведите:

```
Количество заказов: ...
```

Задание 10. Удаление тестового товара

В предыдущих заданиях мы создали товар `Мокка`.

После завершения работы удалите его:

```sql
DELETE FROM products
WHERE name = N'Мокка';
```

Используйте:

```
ExecuteNonQuery()
```

После выполнения убедитесь, что товар удалён.

Самостоятельная часть

Теперь необходимо самостоятельно расширить программу.

Приложение должно вывести в консоль небольшую статистику кофейни.

Реализуйте следующие операции.

1. Количество клиентов

Получите количество записей из таблицы:

```
customers
```

Используйте:

```sql
SELECT COUNT(*)
FROM customers;
```

2. Количество доступных товаров

Получите количество товаров, которые доступны для продажи:

```sql
SELECT COUNT(*)
FROM products
WHERE is_available = 1;
```

3. Самый дешёвый товар

Получите минимальную цену:

```sql
SELECT MIN(price)
FROM products;
```

Используйте `ExecuteScalar()`.

4. Количество выданных заказов

Получите количество заказов со статусом:

```
Выдан
```

SQL:

```sql
SELECT COUNT(*)
FROM orders
WHERE status = N'Выдан';
```

5. Добавление собственного товара

Самостоятельно добавьте в таблицу `products` новый товар.

Выберите:

- название;
- категорию;
- цену.

После добавления выведите количество изменённых строк.

Дополнительное задание

Создайте небольшое текстовое меню:

```
=== COFFEE SHOP ===

1. Количество товаров
2. Список товаров
3. Количество клиентов
4. Количество заказов
5. Самый дорогой товар
6. Добавить товар
0. Выход
```

Пока не требуется реализовывать полноценную систему.

Достаточно использовать:

```
Console.ReadLine();
```

и конструкцию:

```
switch
```

для выбора операции.

Для каждого пункта меню должен выполняться соответствующий SQL-запрос.

Что необходимо продемонстрировать преподавателю

Готовая программа должна:

1. Успешно подключаться к `coffee_shop`.
2. Использовать `SqlConnection`.
3. Использовать `SqlCommand`.
4. Содержать пример `ExecuteScalar()`.
5. Содержать пример `ExecuteNonQuery()`.
6. Содержать пример `ExecuteReader()`.
7. Получать данные из нескольких таблиц базы.
8. Выполнять как минимум одну операцию изменения данных.
9. Выводить результаты выполнения запросов в консоль.

Выводы по работе

В ходе практической работы было создано C#-приложение, взаимодействующее с SQL Server через ADO.NET.

Были освоены основные способы выполнения SQL-команд.

Также была отработана базовая последовательность работы с базой данных.

Полученные знания являются основой для дальнейшей работы с параметризованными SQL-запросами, `SqlDataReader`, транзакциями и асинхронным доступом к базе данных.